# Paso 1 — La idea: cómo nace, qué se mide antes de proponerla y qué se le exige antes de gastar CPU

**Fecha:** 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 1 de `docs/AgentPDFs/WORKFLOW.md`: el perfil de mercado, la memoria, la cola de bibliografía, el tablero, los tres agentes (`researchDirector`, `ideaExpert`, `ideaReviewer`), la skill `/research-direct` y las reglas duras 11, 13, 14 y 15 en lo que tocan a la idea.

Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.

**Qué es:** el documento de consulta del paso 1, con la misma forma que el del paso 16 (`criterios-paso-16-spp-2026-10-07`). Aquí sí hay estadística: el perfil de mercado mide cada regla contra un nulo y sólo sobre el tramo `build`. La explico desde cero, digo qué cuenta como «mirada» a los datos y qué no, repaso pieza a pieza lo que cada módulo calcula y lo que cada agente tiene que entregar, y cierro con lo que no cuadra y con las decisiones que te quedan.

**De dónde salen los números.** Este paso no tiene corrida de SQX. Los ejemplos son:

- el **mapa del perfil** tal como está hoy en disco (`AlgoData/research/profiles/measures.csv` y `scores.csv`, juzgados el 2026-10-08 a las 07:08 por la recarga diaria de bibliografía; `run.json` dice código `7203f4c-dirty` y 16.036 pruebas);
- el **tablero de hoy**, recalculado por mí en memoria con el código actual (`studies.research.board.many.run`), sin escribir nada. El `board.json` en disco es del 2026-10-02 y ya no coincide en el orden;
- la **única propuesta que existe**: `AlgoData/research/proposals/20261002-203224-DJ30-H1-long-ruptura` y su fichero de idea `AlgoData/ideas/DJ30/2026-10-02-ruptura-h1-largo.md`. Se selló con el código del 2-10; donde el código de hoy haría otra cosa, lo digo;
- la **potencia** (`AlgoData/research/power/`) y el **perfil agrupado** (`AlgoData/research/pooled/`), los dos del 2026-10-02 (código `ba0638d-dirty`), anteriores a que el mapa creciera.

Lo que lleva «calculado por mí» es aritmética mía sobre esos ficheros (scripts en la carpeta temporal `scratchpad/paso01/`: `hoy.py` y `mapa.py`); no es un resultado del estudio. «De juguete» quiere decir cuenta hecha a mano para enseñar.

**El commit que describo:** `b2b4c8c`, más el árbol de trabajo. Ficheros del paso con cambios sin confirmar: `.claude/agents/ideaExpert.md` (9 líneas nuevas: el párrafo de la bibliografía), `studies/research/README.md`, `marketProfile/{bibliography,report,series,store}.py`, `marketProfile/measure/rules.py`, `memory/ideas.py`; y **sin seguimiento de git** todavía: `.claude/agents/literatureScout.md`, casi todo `studies/research/literature/`, `marketProfile/literature.py` y `topup.py`, y casi todo `pooledProfile/` y `profilePower/`. Describo lo que hay en el disco.

**Ninguna línea de este documento está congelada.**

---

## 0. En una página

**La pregunta del paso.** ¿Qué regla de entrada merece que el custodio trabaje horas para ella, y qué se sabe de esa regla antes de construir nada?

**Lo que hay cargado hoy.** Ningún criterio numérico propio: el paso 1 no tiene reglas en `pipeline/autopilot/criteria.yaml` ni hechos en `facts.py`. Lo que hay son tres cosas distintas:

1. un **mapa medido** (el perfil), con cuatro filtros fijos en `marketProfile/config.yaml`: significativa tras corregir al 5 %, paga al menos 2 veces el coste, signo positivo en más de la mitad de los años, al menos 40 operaciones al año;
2. una **puerta de entrada al tablero**: tu prior dice Alta, **o** la celda pasa los cuatro filtros, **o** una variante del barrido pasa en meseta;
3. una **lista de negativas** que el código aplica a la propuesta del director (`board/proposal.py`, función `check`): tres ideas, una dirección, hoja de hipótesis, revisión hecha por otro agente, postura escrita ante cada fracaso previo, preguntas con sus lecturas.

Todo lo demás —que la idea tenga mecanismo, que sólo se midiera en `build`, que las miradas estén contadas— son instrucciones a los agentes, no comprobaciones del código.

**Lo que sale hoy.** De 1.064 celdas-familia medidas, **2 pasan los cuatro filtros** (DJ30 H1 largo ruptura y EURUSD M15 corto momentum). Al tablero entran 359: **357 sólo por tu prior** y esas 2 por lo medido. Hay **una propuesta** escrita (DJ30 H1 largo ruptura, tres ideas, 2-10): una idea «tumbada» por el revisor, dos «dudosas», 40 preguntas abiertas, ninguna contestada y nada construido.

**Las doce cosas que conviene saber antes de leer el resto** (las cuatro últimas salen de la revisión del Council y cambian cómo leer las dos celdas que pasan; las cifras marcadas «calculado por el revisor» no son resultado del estudio):

1. **Las miradas de la única propuesta cuentan cero.** El `ideaExpert` midió 68 estadísticas condicionales para DJ30 y lo escribió bajo el título «## 3. Hipótesis medidas en total», que es el que le pide su propia ficha (`.claude/agents/ideaExpert.md:116`). La memoria busca el texto «Hipótesis medidas» seguido de un número (`memory/ideas.py:11`) y ahí no lo encuentra: apunta **0**. El fichero del oro, que escribió «Hipótesis medidas: **24**», sí cuenta (29). Esa cuenta sólo la lee el informe impreso `ledger.report`; ni la ventana, ni el manifiesto del archivo, ni ninguna regla del autopiloto la usan, con el DJ30 o con el oro (leído por el revisor en el código).
2. **Hoy la página que lee el director no trae ninguna celda medida.** Las dos celdas que pasan los filtros están en los puestos 297 y 313 de 359, y la página imprime 40. Los 20 primeros puestos son celdas de prior Alta con evidencia «débil» (significativa y estable, sin pasar los cuatro filtros: 18 pagan menos de 2× y 2 pagan más pero operan menos de 40 al año), empatadas a 67,5 puntos; los 276 siguientes, celdas de prior Alta sin evidencia, empatadas a 60,0 y ordenadas por orden alfabético del activo (AUDJPY ocupa 20 de las 40 líneas).
3. **La familia de pruebas crece cada mañana y el veredicto se mueve con ella.** El 2-10 el mapa tenía 14.364 pruebas; hoy 16.036, porque la recarga diaria añade las reglas que trae el `literatureScout` y vuelve a juzgar el mapa entero. La p más grande que la corrección aún nombra ha bajado de 5/3001 a 4/3001, y la q de la celda del DJ30 ha subido de 0,030 a 0,033 sin que nadie la haya vuelto a medir. Con el rango fijo, dejaría de pasar cuando el mapa supere unas 24.300 pruebas (calculado por mí).
4. **Significativo y rentable casi nunca coinciden.** De las 276 medidas con operaciones, sin reloj, que la corrección nombra, la mediana paga 0,28 veces el coste y sólo 5 llegan a 2×. Y al revés: de las 267 que pagan, son estables y operan lo bastante, sólo 2 tienen un z de 3 o más. El estudio de potencia lo explica: en H1 el perfil sólo podría haber visto una ventaja de 2× el coste, como mucho, en el 14-36 % de sus pruebas para las cinco familias de la tabla del apartado 3.2 (sesión 8 %; patrón no estimada). Es un techo: la cifra viene de una aproximación que, donde se pudo simular (H4), promete más de lo que la simulación detecta, y es del corte del 2-10.

5. **El nulo es un remuestreo barra a barra, con reposición.** Conserva la deriva y la volatilidad media y rompe todo lo demás, incluida la agrupación de la volatilidad. Por eso `volatilidad` puntúa alto en todas partes y no dice nada (lo declara el propio README), y por eso una medida larga en un índice se compara con una nula que ya gana por la deriva. Esa deriva se reparte por igual en el tiempo, no por rachas como en la serie real (apartado 2.2).
6. **El revisor sólo anota.** `review.removes` está en `false` (`memory/config.yaml:172`): una idea «tumbada» se puede lanzar igual en cuanto sus preguntas estén contestadas. Y su ficha pide «40-50 operaciones al año» (`ideaReviewer.md:55-56`) cuando el suelo de trabajo es 35 y el filtro del perfil 40.
7. **«Sólo `build`» está forzado en el perfil y es una instrucción para el experto.** El perfil corta las barras en un único sitio (`marketProfile/inputs.py:36-48`) y un test lo comprueba envenenando lo posterior. El `ideaExpert` tiene todas las herramientas y nada le impide leer `oos1`: sólo su ficha.
8. **La fila del paso 1 en `WORKFLOW.md` sigue diciendo «Idea en el chat | — | —»** (línea 26), y la lista de segmentos dice que `build` lo toca «sólo el paso 6» (línea 141). Todo lo que describe este documento no aparece en la espina dorsal.
9. **Las dos celdas que pasan salen de un grupo de familias sin exceso de p pequeñas.** En tendencia, ruptura y momentum juntas hay 7 pruebas con p ≤ 4/3001 y el azar solo daría unas 9; corregidas sólo contra su propio grupo, no pasa ninguna de las dos. Pasan porque BH sube el listón de todas por las 360 nombradas de reversión y patrón (calculado por el revisor; apartado 5.15).
10. **El veredicto de la celda del DJ30 no es reproducible.** Pasa con 2 mundos nulos por encima del real y el corte admite 3: volver a medir con otra semilla la dejaría fuera entre una vez de cada tres y una de cada dos (calculado por el revisor; apartado 5.16). Y las pruebas vecinas se parecen tanto que las 16.036 equivalen a bastantes menos (apartado 5.18).
11. **El múltiplo que enseña una celda al pasar viene inflado.** El 3,14× del DJ30 no estima lo que pagará: con ventaja verdadera de 0, 1× o 2× el coste se vería entre 2,8× y 3,1× al pasar (calculado por el revisor; apartado 5.17).
12. **«Sólo `build`» no corta una fuga.** La prior (escrita en octubre de 2026), la bibliografía posterior a 2019 y el propio agente traen información de los años de `oos1` y `oos2` sin abrir ningún fichero de ellos (apartado 5.19).

---

## 1. Qué es el paso 1 y cómo decide

### 1.1 Dónde está

Es el primer paso: antes de él no hay nada, y después vienen el vocabulario (2), la plantilla (3), el preflight (4), el proyecto (5) y el build (6). El código vive en `studies/research/` (seis carpetas) y los agentes en `.claude/agents/`. Nada de este paso toca SQX.

### 1.2 Las dos formas de llegar a una idea

**Con pista tuya.** Dices una idea en el chat. El `ideaExpert` la convierte en regla exacta, o `/sqx-strategy-template` pregunta la lógica y autora la plantilla. Aquí manda la regla 11: cada frase con dos lecturas es una pregunta.

**Sin pista, por el director** (`/research-direct` o el botón «Proponer investigación»). Es una cadena de seis pasos (`board/proposal.py:15-17`):

| # | quién | qué hace | coste |
|---|---|---|---|
| — | Python | refresca la memoria ( `memory.report` ) e imprime el tablero ( `board` ) | 0 tokens, ~50 ms |
| 1 | `researchDirector` | lee la página del tablero | una sesión Opus para toda la cadena |
| 2 | `researchDirector` | elige UNA celda (activo × marco × dirección × familia) y la justifica con las filas del perfil | |
| 3 | `ideaExpert` | tres reglas distintas dentro de esa familia, cada una con su hoja de hipótesis; mide sobre `build` y cuenta lo que mide | |
| 4 | `ideaReviewer` | intenta tumbar cada idea por cuatro frentes; no mide nada | |
| 5 | `researchDirector` | una paleta de bloques por idea | |
| 6 | Python | comprueba el borrador, lo sella y lo guarda ( `board.proposal` ) | 0 tokens |

Después decides tú: vetas, contestas las preguntas y lanzas desde la ventana (BIBLIOTECA › Investigar). La skill y el director no lanzan nada.

### 1.3 Qué es una celda

Un activo, un marco temporal, una dirección y una familia de comportamiento: «DJ30 · H1 · largo · ruptura». Hay 19 activos × 4 marcos (M15, M30, H1, H4) × 2 direcciones × 7 familias = **1.064 celdas-familia**. Las familias son tendencia, ruptura, reversión, momentum, volatilidad, patrón y sesión. Sesión y patrón no llegan al tablero porque tu prior no las cubre ( `prior.py:20` ) y ninguna pasa los cuatro filtros. El filtro de reloj de `board/many.py:31` existe, pero hoy no quita ninguna celda: `needs_clock` es falso en las 1.064 (sesión lidera con `prev_day_break`, que no está marcada como de reloj; véase la decisión 12 del apartado 6.3).

### 1.4 Qué lee cada pieza

| pieza | lee | no lee |
|---|---|---|
| perfil, barrido, potencia, agrupado | las barras M1 del tramo `build`, cortadas antes de remuestrear | `oos1`, `oos2` |
| memoria | registros de proyectos, corridas del autopiloto, ficheros de idea | precios |
| tablero | `scores.csv`, el `best.csv` del barrido, la memoria, tu prior | precios |
| `ideaExpert` | lo anterior y, por instrucción, sólo `build` | por instrucción, `oos1` / `oos2` |
| `ideaReviewer` | texto: la idea, la memoria impresa, la ficha del activo | cualquier fichero de datos (no tiene consola) |

### 1.5 Cómo decide

No hay PASA / LIMBO / NO PASA. Hay tres niveles de «no»:

- **El perfil no deja entrar la celda** en el tablero, salvo que tu prior diga Alta.
- `proposal.check` **no acepta el borrador**: el director tiene que rehacerlo.
- **Tú vetas**, o dejas una pregunta sin contestar: esa idea no es lanzable ( `proposal.py:98-106` ).

### 1.6 Las tres ideas que nos acompañan

Son las de la única propuesta, las tres para DJ30 · H1 · largo · ruptura:

| | A | B | C |
|---|---|---|---|
| nombre | `donchian55BreakTightRange` | `donchian55BreakUptrendD1` | `dayOpenNoiseBreakUp` |
| regla, en corto | cierre sobre el máximo de 55 barras **y** rango de 55 barras ≤ 9 ATR | cierre sobre el máximo de 55 barras **y** diario sobre su media de 200 | cierre sobre la apertura del día más un múltiplo del ATR |
| cantera | propia | perfil ( `channel55_up200` ) | libros (Zarattini, Aziz y Barbon, 2025) |
| hoja: operaciones al año | 45 | 60 | 40 |
| hoja: duración mediana | 20 h | 20 h | 6 h |
| revisor | dudosa | dudosa | **tumbada** |
| preguntas abiertas | 16 | 13 | 11 |
| bloque que hay que autorar | `ChannelWidthBelowATR` | `DailyCloseAboveDailySMA` | `CloseCrossAboveDayOpenPlusATR` |
| horas de custodio estimadas | 0,8 | 0,8 | 0,8 |

A y B llevan dos condiciones fijas y ningún hueco aleatorio (regla 14: dos fijas agotan el tope). La primera pregunta de C es si «la apertura del día» cuenta como bloque de calendario bajo la regla 15.

---

## 2. Fundamentos de este paso

### 2.1 Una estadística condicional y su t

Una «medida» del perfil es una regla sencilla: «cuando pase X, compro en la apertura siguiente y salgo N barras después». Lo que se mira es el rendimiento de esas operaciones.

**De juguete.** Cinco operaciones dan, en puntos: +3, −1, +2, +4, −3.

```
media               = (3 − 1 + 2 + 4 − 3) / 5           = 1,0
desviaciones        = 2, −2, 1, 3, −4      cuadrados = 4, 4, 1, 9, 16     suma = 34
desviación típica   = √(34 / 4)                           = 2,92
error estándar      = 2,92 / √5                           = 1,30
t                   = media / error estándar = 1,0 / 1,30  = 0,77
```

El t dice cuántos errores estándar separan la media de cero. Con 0,77 no hay nada que decir: cinco operaciones al azar dan eso o más una vez de cada cuatro o cinco.

El perfil usa exactamente esto: el t de los rendimientos en logaritmos, de apertura a apertura ( `measure/trades.py:26-36` ), y devuelve 0 si hay menos de 30 operaciones. Usa el t y no la media a secas a propósito: al remuestrear, los rendimientos cambian de hora y de régimen, y una media se compararía con una dispersión que no es la suya.

**Un matiz que importa luego.** En las medidas originales cada barra con señal abre una operación, aunque la anterior siga abierta (`measure/trades.py:22-23`): las operaciones se solapan y no son independientes. Por eso el t no se lleva a una tabla: se compara con el t de la misma regla en series remuestreadas, que tienen el mismo solape. Las 31 base y 4 del ensanche con etiqueta `horizon` (`look100_hold24`, `look200_hold48`, `channel100_hold24`, `channel200_hold48`) solapan; las del tipo `rule` (34 del ensanche, contexto y salida, más 33 de bibliografía) llevan una posición a la vez.

### 2.2 El nulo: qué es «por azar» aquí

Un **nulo** es un mundo fabricado en el que, por construcción, la regla no puede tener ventaja. Se aplica la misma regla en muchos de esos mundos y se mira cuántas veces sale un resultado tan bueno como el real.

**Cómo lo fabrica el perfil.** Cada barra se guarda como cuatro distancias al cierre anterior (apertura, máximo, mínimo, cierre; `series.py:123-126`). Un mundo nulo se hace sacando barras **al azar y con reposición** de la serie real y encadenándolas (`series.py:136-151`, `engines/resample/draws.py:84-100`, con bloque de 1 barra en los cuatro marcos, `config.yaml:23`). El calendario —la hora y el día de cada posición— se queda quieto.

**De juguete.** Serie real de seis barras, con sus cierres relativos: +2, −1, +3, −2, +1, +4. Un sorteo saca las posiciones 3, 3, 6, 1, 5, 2: la serie nula es +3, +3, +4, +2, +1, −1. Dos cosas a notar:

- la barra 3 sale dos veces y la 4 ninguna. No es «las mismas barras en otro orden»: es una muestra con reposición. En media conserva la deriva y la volatilidad; en un sorteo concreto, no exactamente;
- cualquier relación entre una barra y las siguientes ha desaparecido. Si la regla funciona porque «después de X suele venir Y», en el mundo nulo no puede funcionar.

**Lo que el nulo conserva y lo que rompe:**

| conserva | rompe |
|---|---|
| la deriva media (un índice sigue subiendo) | que tras una ruptura siga la tendencia |
| el tamaño medio de las barras | que la volatilidad venga en rachas |
| la forma de cada barra (mecha, cuerpo) | que una hora del día se mueva más que otra |

La primera fila explica por qué la nula de una medida larga en un índice no está en cero: en el DJ30 la nula de `channel55_up200` tiene media 0,60. Comprar al azar en un índice que sube ya da un t positivo. Lo que se juzga es el **exceso sobre eso**, pero sólo sobre la deriva media repartida por igual en el tiempo: el nulo conserva la deriva media, no cuándo ocurrió. Un índice sube por rachas, y una regla larga que sólo opera en la racha buena (por ejemplo con el diario sobre su media de 200) le gana a un nulo que reparte la subida por igual, sin que la entrada aporte nada. `channel55_up200` sólo opera con el diario sobre su media de 200, y encaja con lo que recoge el apartado 3.1.6: la hermana sin filtro no pasa, el 86 % de las rupturas ya estaban sobre esa media (3.7) y 2014-2015 dan 2,83 y 1,59 frente a 17-24 en 2016-2019.

La segunda fila tiene un precio declarado: las medidas sin dirección de la familia `volatilidad` se contrastan contra «la volatilidad no se agrupa», que es falso en todos los mercados. Salen significativas en todas partes y no informan (`marketProfile/README.md`, «The block length»).

**Por qué bloque de 1.** Se midió el 1-10 (`blocklen.py`): con bloques más largos el nulo hereda la estructura que quiere romper y rechaza de menos (0,3-0,8 % de falsas alarmas al 5 % con 24 barras, frente a 4,0-7,5 % con 1).

Esa comprobación se hizo al 5 %, con 200 sorteos (`blocklen.py:74-81`: la p mínima posible era 1/201, así que no podía ver la cola de 0,00133), en 6 celdas (XAUUSD, EURUSD y USA500 en H1 y H4), con copias de signo a cara o cruz y con las 31 medidas originales. El perfil decide con $p \le 0{,}00133$, y a ese nivel las falsas alarmas del

nulo no están medidas: un nulo que rompe la agrupación de la volatilidad puede acertar al 5 % y fallar por varias veces en la cola. En XAUUSD H1 el tamaño medido es 7,45 %, donde el nominal es 5 %. Las medidas añadidas desde el 2-10 (hoy 110 en total) no se han comprobado ni al 5 %. El único dato cerca del corte real es el de `power/curve.csv` con efecto 0: 1 falsa alarma de 24 ( `run3_follow` , H4) donde se esperan 0,4, que no decide nada. Todo esto, calculado por el revisor sobre `blocklen.csv` y `power/curve.csv` .

### 2.3 El p-valor por conteo, y el z

```
p = (1 + nº de mundos nulos con estadístico ≥ el real) / (nº de mundos + 1)        one.py:89-92
z = (real − media de los nulos) / desviación típica de los nulos                    one.py:158
```

**De juguete.** El real da t = 2,1. Se hacen 19 mundos nulos y uno de ellos da 2,1 o más: p = (1 + 1) / (19 + 1) = 0,10.

El «+1» de arriba y de abajo cuenta la serie real como un mundo más. Consecuencia: con 3.000 sorteos ( `config.yaml:13` ) **la p más pequeña posible es 1/3001 = 0,00033**, y las p van a saltos de 1/3001. La p es además una estimación con 3.000 sorteos y su ruido llega al veredicto de una celda en el corte (apartado 5.16).

El p sólo mira hacia arriba (¿cuántos nulos igualan o superan al real?). Para el lado corto se da la vuelta a la serie y se mide igual. Una p de 0,95 o más quiere decir que el real queda por debajo de casi todos los nulos: la regla va al revés.

El z es la misma comparación en otra unidad. El z no decide nada: sólo alimenta la puntuación 0-100, que ya no ordena nada (apartado 3.1.7).

Una medida con menos de 30 operaciones no se contrasta: se le pone p = 1 ( `one.py:151,159` ).

### 2.4 Muchas pruebas a la vez: Benjamini-Hochberg

Con una sola prueba, p ≤ 0,05 quiere decir «por azar pasaría una vez de cada veinte». Con 16.036 pruebas, por puro azar saldrían unas **800** con p ≤ 0,05 (unas 690 si no se cuentan las 2.178 que no se contrastaron: 13.858 × 0,05, calculado por el revisor). En el mapa real hay 1.319 (calculado por mí): la mayoría podría ser ruido.

**Benjamini-Hochberg** (BH) no pide que no haya ningún falso positivo; pide que, **de las que nombres, como mucho el 5 % sean falsas en promedio**. Eso se llama controlar la tasa de falsos descubrimientos (FDR).

**De juguete, con diez p ordenadas.** El listón de la i-ésima es i/10 × 0,05:

| puesto | p | listón | ¿p ≤ listón? |
|---|---|---|---|
| 1 | 0,001 | 0,005 | sí |
| 2 | 0,004 | 0,010 | sí |
| 3 | 0,012 | 0,015 | sí |
| 4 | 0,030 | 0,020 | no |
| 5 | 0,040 | 0,025 | no |
| 6-10 | 0,20 … 0,90 | 0,030 … 0,050 | no |

Se busca el **último** puesto que cumple (el 3) y se nombran todas hasta él: tres. Sin corregir habrían sido cinco.

La **q** es la p ya corregida: la FDR más pequeña a la que esa prueba saldría nombrada. Se calcula como p × (nº de pruebas) / (puesto), sin dejar que suba al bajar de puesto. Aquí: 0,010 · 0,020 · 0,040 · 0,075 · 0,080. «q ≤ 0,05» y «nombrada» son lo mismo. El código es `engines/inference/fdr.py:10-25` (las nombradas) y `:28-44` (la q).

**El supuesto de BH.** BH garantiza su 5 % si las pruebas son independientes o se mueven juntas en el mismo sentido, y si la lista de pruebas está fijada antes de mirar. Aquí las pruebas vecinas se parecen mucho (la misma regla en marcos contiguos del mismo activo da casi el mismo z: correlación mediana 0,83, calculado por el revisor) y la lista crece cada mañana. La media sigue acotada, pero en un mapa concreto las falsas vienen en racimo (apartado 5.18).

**Dos consecuencias que vuelven en el apartado 5:**

- el listón depende de **cuántas pruebas hay**. Añadir pruebas al mapa cambia el veredicto de las que ya estaban;
- con p a saltos de 1/3001, muchas pruebas empatan en el mismo valor y el corte cae entre dos saltos.

### 2.5 Qué quiere decir «pagar», «estable» y «frecuente»

Las tres se calculan sobre las barras reales, no contra el nulo ( `one.py:43-61` , `many.py:73-78` ):

- **Efecto**: la media, en unidades de precio, de (apertura de salida − apertura de entrada), con el signo del lado. Es bruto.
- **Coste**: el de una ida y vuelta en el tramo `build` : spread + 2 × deslizamiento + comisión, de la ficha del activo ( `inputs.py:73-90` ). El swap no está.
- **Múltiplo** = efecto ÷ coste. **Paga** si múltiplo ≥ 2 ( `config.yaml:27` ).
- **Estable** si el efecto medio es positivo en **más de** la mitad de los años naturales en que la regla operó ( `config.yaml:28` ). Con 6 años hacen falta 4; con 7, también 4.
- **Frecuente** si hace al menos 40 operaciones al año ( `config.yaml:31` ); el código se niega a bajar de 35 para ningún activo ( `many.py:10,13-30` ).

**De juguete, la estabilidad.** Siete años con efecto medio +5, +2, −1, +8, +3, −4, +6: cinco positivos de siete = 0,71 > 0,5, estable. Ojo a lo poco que pide: una regla sin nada dentro tiene, por simetría, una probabilidad cercana al 50 % de sacar 4 o más años positivos de 7; y en un índice con deriva, bastante más.

### 2.6 Potencia: lo que una prueba no puede ver

Que una regla no salga significativa no quiere decir que no tenga ventaja: puede que la prueba no tenga fuerza para verla. La **potencia** es la probabilidad de que la prueba nombre una ventaja que existe de verdad. El **efecto mínimo detectable** (MDE) es el tamaño de ventaja a partir del cual la prueba la nombra el 80 % de las veces.

Aquí pesa mucho, porque el listón tras corregir es exigente. La p más grande que BH nombra hoy es 4/3001 = 0,00133; eso equivale a un z de **3,0** (para una sola prueba al 5 % bastaría 1,65).

**De juguete.** Una regla con 60 operaciones al año durante 6 años son 360 operaciones. Si cada operación tiene una dispersión de 10 costes, el error estándar de la media es 10/√360 = 0,53 costes. Para llegar a z = 3 por encima del nulo hace falta una media de unos 1,6 costes; para verlo el 80 % de las veces, unos 2 (hay que sumar 0,84 errores estándar: 3,84 × 0,53). Con 10 operaciones al año (60 en total) el error es 1,29 y hacen falta unos 5 costes. **La potencia vive del número de operaciones.**

Con potencia baja, además, lo que pasa viene inflado: una regla sólo cruza un listón de z = 3 si además de su ventaja ha tenido suerte, así que el múltiplo que enseña al pasar es mayor que el verdadero (la «maldición del ganador»; apartado 5.17).

### 2.7 Qué es una «mirada» y por qué se cuenta

Una **mirada** es cada vez que se calcula un número sobre los datos y ese número puede influir en lo que se decide después. Cada mirada es una ocasión más de encontrar algo por casualidad.

**De juguete.** Si miras 21 condiciones que no tienen nada dentro y das por buena la que supere un listón del 5 %, la probabilidad de que al menos una lo supere es $1 - 0{,}95^{21}$ = **66 %**. Con 68 miradas, **97 %**, y lo esperado son 3,4 falsas (calculado por mí; supone miradas independientes, que no lo son del todo: variantes de la misma ruptura se parecen, así que la cifra real es algo menor).

**Dónde se paga esa cuenta en este proyecto.** En el Sharpe desinflado del ledger. El listón de ese cálculo es el mejor Sharpe que cabría esperar por puro azar tras N intentos, que crece aproximadamente como σ × √(2 × ln N):

```
N = 100      →  √(2 × 4,61)  = 3,03 σ
N = 10.000   →  √(2 × 9,21)  = 4,29 σ
```

`ledger.report` suma a la N de un estudio las hipótesis que el `ideaExpert` midió para esa celda (`ledger/report.py:91`, `memory/trials.py:18-43`).

**Qué cuenta y qué no, tal como está en el código:**

| cosa | ¿es una mirada a los datos? | ¿se suma a la N? |
|---|---|---|
| cada estadística condicional que mide el `ideaExpert` | sí | sí, la cifra que escriba en su fichero, una vez por fichero (`trials.py:41`) |
| cada estrategia que el builder puntúa | sí | sí (es la N de siempre) |
| las pruebas del perfil y las variantes del barrido en esa celda | sí | **no**; se imprimen al lado, «aparte, sin sumar» (`trials.py:46-61`), como una sola cifra: para DJ30 H1 son 904 miradas = 202 pruebas del perfil en ese lado (101 largo + 101 corto; las 9 sin dirección no cuentan, no 211) + 702 variantes del barrido (13 × 3 × 9 × 2); desglose calculado por el revisor con `trials.profile_looks`. Se corrigen entre ellas con BH |
| el director leyendo `measures.csv` para elegir celda | lee números ya calculados | no |
| la revisión del `ideaReviewer` | no: sólo texto | no, y por diseño |
| las variantes que probaron los autores de un paper | sí, de ellos y sobre sus datos | no; se anota en la entrada (`selection.variants_tried`) y en la línea `Origen:` |
| tu prior | no medida aquí | no |

Dos reglas de reparto: largo y corto no comparten cuenta, y una idea que aún no es plantilla (sin familia) cuenta en todos los estudios de su activo, marco y dirección.

### 2.8 Por qué sólo el tramo `build`

Cada activo tiene tres tramos: `build` (lo que el generador ve), `oos1` (pasos 7 a 16) y `oos2` (reservado para 17 a 20). Si una idea se elige mirando `oos1`, el retest de `oos1` deja de ser fuera de muestra para esa idea: ya se sabe que ahí funciona, porque por eso se eligió. El coste no se ve en ningún número; simplemente los pasos 8 a 16 dicen que sí con más facilidad de la que deberían.

Hay además una fuga que cortar las barras no impide: la prior, la bibliografía y el propio agente traen información de los años de `oos1` y `oos2` (apartado 5.19).

Lo que `build` no arregla: una idea elegida mirando `build` **sí** está seleccionada dentro de muestra. El build de SQX la va a confirmar casi seguro, y eso no es evidencia. La evidencia empieza en el paso 7. Por eso el tablero imprime al pie «una variante del barrido es una hipótesis elegida dentro de muestra, no un hallazgo», y lo mismo vale para cualquier celda que pase «desnuda».

### 2.9 El factor «pasado» del tablero: una tasa con pocos datos

Si una familia se probó 3 veces en una clase de activo y no sobrevivió ninguna, decir «tasa 0 %» es demasiado. El tablero usa una tasa **encogida** hacia la media de todo lo probado ( `board/factors.py:21-48` ):

```
tasa global = (corridas con supervivientes + 1) / (corridas cerradas + 2)
a = tasa global × 4 + supervivientes de esta familia y clase
b = (1 − tasa global) × 4 + fracasos de esta familia y clase
tasa = a / (a + b)
```

El 4 son «pseudointentos» ( `board/config.yaml:35` ): hasta que no hay más de cuatro corridas cerradas, manda la media global.

**De juguete.** Nada cerrado en ningún sitio: global = 1/2. Familia sin probar: a = 2, b = 2, tasa 0,50, intervalo del 90 % de 0,14 a 0,86. Con 0 de 3: a = 2, b = 5, tasa 0,29 en lugar de 0.

**Hoy** no hay ninguna corrida cerrada con familia (apartado 3.3), así que el factor vale 0,50 en las 359 celdas: es una constante y no ordena nada.

---

## 3. Lo que hace o calcula, pieza a pieza

### 3.1 El perfil de mercado ( `studies/research/marketProfile/` )

**3.1.1 Entrada.** Las barras M1 del activo, cortadas a `[inicio de build, fin de build)` en `inputs.py:36-48` , y luego remuestreadas a M15, M30, H1 y H4. `tests/test_marketprofile.py:109` ( `test_build_only` ) envenena las barras posteriores al fin de `build` y comprueba que ninguna medida se mueve.

**3.1.2 Las medidas.** 110 nombres distintos hoy. Las que abren operaciones se miden dos veces (largo y corto); las que no tienen dirección, una. Salen **211 pruebas por celda** (activo × marco) y 16.036 en las 76 celdas. Por etiqueta (calculado por mí sobre `measures.csv` ):

| etiqueta | pruebas | qué son |
|---|---|---|
| `base` | 4.028 | las 31 originales |
| `horizon`, `hold`, `exit`, `context`, `regime` | 5.320 | el ensanche del 2-10 |
| `literature` | 6.688 | reglas de la bibliografía, las de mano y las del scout |

608 pruebas son de reloj (se miden y nunca puntúan ni llegan al tablero) y 2.178 tienen p = 1 puesta a mano por tener menos de 30 operaciones (`one.py:151,159`). Otras 454 de las 2.632 con p = 1 sí se contrastaron y dieron p = 1 (el real quedó por debajo de los 3.000 nulos): 241 son medidas sin entrada y 213 tienen 30 operaciones o más (calculado por el revisor sobre `measures.csv`).

**3.1.3 El contraste.** Lo de los apartados 2.2 y 2.3: 3.000 mundos nulos por celda, p por conteo, z.

**3.1.4 La corrección.** BH al 5 % sobre **todas** las pruebas del mapa juntas (`many.py:70`). Al lado, y sin decidir nada, la misma corrección dentro de cada familia (`q_family`, `passes_family`; `many.py:71-72`).

**3.1.5 Los cuatro filtros y el veredicto de una medida.** `passes` = significativa **y** paga **y** estable **y** frecuente (`many.py:77-78`).

**3.1.6 El ejemplo real: `channel55_up200`, DJ30 · H1 · largo.** Recalculado por mí desde la fila de `measures.csv`:

| | valor | cuenta |
|---|---|---|
| operaciones en `build` | 445 | |
| operaciones al año | 71,2 | 445 ÷ 6,25 años (30-09-2013 a 31-12-2019) |
| t real | 3,816 | |
| nulo: media y desviación | 0,601 y 1,043 | la media no es cero: la deriva del índice |
| z | 3,08 | (3,816 − 0,601) / 1,043 |
| p | 0,0010 = 3/3001 | **2** de 3.000 mundos nulos igualaron o superaron al real |
| puesto entre las 16.036 | 486 | hay 486 pruebas con p ≤ 3/3001 |
| q | 0,0330 | 0,0010 × 16.036 / 486 |
| efecto medio | 16,37 puntos de índice | |
| coste | 5,21 puntos | spread 2,60 + deslizamiento 2 × 1,30 + comisión 0 |
| múltiplo | 3,14 | 16,37 / 5,21 |
| años con signo | 6 de 6 | 2014: 2,83 · 2015: 1,59 · 2016: 24,14 · 2017: 17,70 · 2018: 22,99 · 2019: 18,18 |
| veredicto | pasa | las cuatro |

Tres cosas que la tabla enseña:

- **2014 y 2015 dan 2,83 y 1,59 puntos, menos que el coste de 5,21.** El signo es positivo y el filtro de estabilidad los cuenta a favor. La regla es «más de la mitad de los años con signo positivo», no «más de la mitad de los años pagando el coste».
- **Son 6 años y no 7** porque el filtro del diario necesita 200 días de historia y en 2013 no opera. Las medidas vecinas tienen 7, y el trimestre de 2013 cuenta como un año entero en su votación.

- **La hermana sin filtro, `channel55`, no pasa:** 907 operaciones (145 al año), múltiplo 2,37, 7 de 7 años, p = 20/3001 = 0,0067, q = 0,149. Paga, es estable y frecuente; no sobrevive a la corrección.

**3.1.7 De la medida a la familia.** Para cada celda y familia (`many.py:98-143`):

- el **líder** es la medida con operaciones que mejor paga entre las que pasan; si ninguna pasa, la de q más pequeña (`many.py:82-88`). Una medida de reloj nunca lidera mientras la familia tenga otra (`many.py:91-95`);
- la familia **pasa** si alguna de sus medidas con operaciones pasa;
- la **puntuación** 0-100 es 100 × media de los z recortados a [0, 5] ÷ 5 (`many.py:129`). El README avisa de que «ya no ordena nada». Medias de hoy por familia: volatilidad 23,5 · patrón 21,0 · sesión 19,2 · reversión 16,3 · momentum 9,6 · tendencia 5,5 · ruptura 4,6. Las dos celdas que pasan puntúan 16,7 y 21,7.

**3.1.8 Lo que sale hoy, entero** (calculado por mí sobre `measures.csv` y `scores.csv`):

| | **cuántas** |
|---|---|
| pruebas | 16.036 |
| con p cruda ≤ 0,05 | 1.319 (el azar solo daría unas 800; unas 690 sin las 2.178 no contrastadas, calculado por el revisor) |
| nombradas por BH | 504. El 5 % (unas 25 falsas) es el promedio sobre la lista entera, que son casi todas reversión a muy corto plazo; no vale para las 2 que además pagan. En el escalón de p de la celda del DJ30 (3/3001 o 4/3001) hay 41 pruebas y el azar solo pondría 8,9: una de cada cinco de ese escalón sería falsa aunque todo lo demás fuera verdad (calculado por el revisor) |
| por familia | reversión 247 · patrón 113 · volatilidad 76 · sesión 61 · momentum 6 · ruptura 1 · tendencia 0 |
| p más grande nombrada | 4/3001 = 0,00133 |
| empatadas en la p mínima (1/3001) | 429 |
| medidas con operaciones, sin reloj, nombradas | 276; mediana de su múltiplo, **0,28**; 222 pagan menos de medio coste |
| nombradas que además pagan ≥ 2× | 5 |
| de esas 5, las que operan ≥ 40 veces al año | **2** |
| celdas-familia que pasan | **2 de 1.064** |

Las dos celdas que pasan pertenecen a ruptura y momentum, y esa lectura tiene un reparo grande: en tendencia, ruptura y momentum juntas hay 7 pruebas con p ≤ 4/3001 y el azar solo daría unas 9; pasan la corrección porque BH sube el listón de todas cuando hay muchas nombradas (apartado 5.15).

Las tres que se caen por frecuencia: AUDUSD H4 corto `extreme3` (9,4× el coste, 15 operaciones al año), GBPJPY H1 largo `bar3atr` (3,9×, 13) y XAGUSD H4 largo `bar2atr_up200` (4,9×, 5).

Con la corrección alternativa por familia pasarían 3 medidas, todas de reversión: EURUSD M15 y M30 corto `day_down_volhigh_1d` (8,1×, 46 al año) y USDCAD H4 largo `down3_up200_sma5` (3,1×, 43). Ninguna de las dos del veredicto oficial está entre ellas: son listas distintas.

**3.1.9 El barrido** ( `sweep.py` , `sweepjudge.py` ). Estudio aparte, con su propia corrección: 13 entradas × 3 parámetros cercanos × 9 salidas × 2 lados = 702 variantes por celda, 1.000 sorteos. Una variante está «en meseta» si al menos 2 vecinas a un paso también pagan ( `config.yaml:88` ). Resultado en disco (2-10, `sweep/counts.csv` ): con alguna variante que pasa, una celda-familia en M15, una en M30 y una en H1; **en meseta, ninguna**. De 760 celdas-familia barridas, 148 no llegan a medio coste con ninguna variante: el tablero las marca «medido en contra».

**3.1.10 Otros ficheros.** `context.csv` (deriva, coste sobre ATR, eficiencia de Kaufman por celda), `favourable.csv` y `avoid.csv` (lo que alimenta `assets/FAMILIAS.md` ), `overlap.csv` (cuánto disparan dos familias en las mismas barras). Son contexto: ninguno mete una celda en el tablero.

### 3.2 La potencia del perfil y el perfil agrupado

`profilePower/` planta una ventaja de tamaño conocido en una copia de la serie sin dirección (cada barra con el signo echado a cara o cruz) y le pasa el contraste del propio perfil. Sus salidas son del 2-10, con el corte de entonces (5/3001 = 0,00167; hoy el corte es más exigente, así que la potencia de hoy es algo menor que estas cifras).

Efecto mínimo detectable, simulación en H4 ( `mde.csv` , 24 réplicas por medida y tamaño de efecto, en 7 tamaños: 0 · 0,5 · 1 · 2 · 5 · 10 · 25; un MDE de «18,5» es una interpolación entre 10× y 25×), en costes brutos por operación:

| medida | operaciones al año | MDE para salir significativa | MDE para pasar los cuatro filtros |
|---|---|---|---|
| `run3_follow` (patrón) | 228 | 1,2 | 4,2 |
| `narrow_break` (volatilidad) | 65 | 7,6 | 7,6 |
| `bar2atr` (momentum) | 16 | 16,0 | no se alcanza ni con 25 |
| `channel55` (ruptura) | 40 | 18,5 | 20,3 |
| `extreme2` (reversión) | 37 | 18,7 | no se alcanza |
| `rsi2_up200_sma5` (reversión) | 42 | 19,0 | no se alcanza |
| `tsmom_d60_run` (tendencia) | 7 | no se alcanza | no se alcanza |

Ensanchado a todo el mapa con la aproximación normal ( `reach.csv` ): la fracción de pruebas que **podrían** haber nombrado una ventaja de 2× el coste.

| familia | M15 | M30 | H1 | H4 |
|---|---|---|---|---|
| momentum | 62 % | 37 % | 23 % | 10 % |
| reversión | 55 % | 45 % | 31 % | 8 % |
| ruptura | 50 % | 29 % | 14 % | 3 % |
| tendencia | 45 % | 35 % | 25 % | 8 % |
| volatilidad | 39 % | 38 % | 36 % | 12 % |

Esta tabla es un techo, no una medida: sale de la aproximación normal ( `analytic.py` ), y `power/calibration.csv` la compara con la simulación y encuentra que promete de más en todos los tamaños (diferencia media −0,035 a 2×, −0,18 a 5×, −0,19 a 10×; `channel55` H4 a 5×: detecta el 12,5 % y la aproximación dice 33,8 %; calculado por el revisor sobre ese CSV; lo reconoce `profilePower/POSSIBLE_IMPROVEMENTS.md:28-29` ). La simulación es sólo H4, sobre copias sin dirección de XAUUSD, EURUSD y USA500 (ningún DJ30), con el corte del 2-10. En H1 la columna de sesión da 8 % y patrón no aparece en `reach.csv` .

Lectura: el filtro pide 2× el coste, pero en H4 el contraste casi nunca puede ver menos de 7-18×. «No pasa» en H4 quiere decir casi siempre «no se puede saber», no «no hay nada».

`pooledProfile/` intenta recuperar esa potencia juzgando cada medida por **grupo** (clase de activo × marco × dirección), con un nulo que remuestrea a la vez todos los activos del grupo. Salida del 2-10 ( `run.json` ): 2.543 pruebas de grupo, 142 nombradas; traerían 218 celdas al tablero, 148 nuevas, pero **0 nuevas** con los cuatro filtros estrictos sobre el efecto encogido. En 200 mundos placebo el procedimiento no nombró ningún grupo. Sólo cuenta: no toca la puerta del tablero.

### 3.3 La memoria ( `studies/research/memory/` )

Recalcula cada vez, desde los registros, qué se probó en cada celda, dónde murió y por qué. Lo que el paso 1 usa de ella:

- **`ideas.index()`** ( `ideas.py:54-68` ): cada idea de cada fichero de `AlgoData/ideas/` , leída por el encabezado «### Idea N — `nombre` · TF · largo|corto» ( `ideas.py:10` ), con las hipótesis medidas del fichero ( `ideas.py:11` : la mayor cifra tras «Hipótesis medidas») y la línea `Origen:` ( `ideas.py:12` ).
- **`queries.failures(…)`** : las corridas cerradas de la celda con causa tipada. Son las que el `ideaExpert` tiene que adoptar o rechazar por escrito.
- **`trials.spent(…)`** : las hipótesis que se suman a la N del ledger.
- **`sheet.py`** : la hoja de hipótesis, su sello (un hash de 16 caracteres de sus cuatro campos, `sheet.py:33-37` ) y la comparación con lo que el build hizo ( `sheet.py:78-99` ): si las operaciones al año o la duración construidas difieren de las predichas en más de un factor 2 ( `memory/config.yaml:166` ), la corrida se cierra con causa `hypothesis_mismatch` .

**Hoy** (recalculado por mí con el código actual):

| idea | activo · marco · lado | hipótesis medidas que cuenta la memoria | origen |
|---|---|---|---|
| `atrImpulseBarUp` | XAUUSD H4 largo | 29 | sin línea `Origen:` |
| `atrImpulseBarDownShort` | XAUUSD H4 corto | 29 | sin línea |
| `squeezeReleaseBBUp` | XAUUSD H1 largo | 29 | sin línea |
| `donchian55BreakTightRange` (A) | DJ30 H1 largo | **vacío** | sin línea |
| `donchian55BreakUptrendD1` (B) | DJ30 H1 largo | **vacío** | sin línea |
| `dayOpenNoiseBreakUp` (C) | DJ30 H1 largo | **vacío** | bibliografía MOM-21b |

Ninguna de las seis es plantilla todavía. De 44 corridas de proyecto que ve la memoria, **34 no tienen ni dirección ni familia** (sin plantilla reconocida no hay familia; no he desglosado cuántas son de Estrategia Simple; el CSV guardado, del 2-10, tenía 62 filas, véase 5.13) y ninguna corrida con familia está cerrada: 0 fracasos que adoptar en cualquier celda.

### 3.4 La cola de bibliografía ( `studies/research/literature/` )

El `literatureScout` busca cada mañana en arXiv y OpenAlex, rechaza cada paper por nueve filtros duros y extrae las reglas que quedan como entradas, cada campo anclado a una cita literal. De los filtros, cuatro son tus reglas duras aplicadas en la puerta: `clock` (regla 15), `not_sqx_at_market` (regla 15, órdenes stop/limit),

`conditions` (regla 14: más de dos condiciones de entrada) y `two_legs` (regla 13). Una regla con dos lecturas lleva una pregunta y no se mide hasta que contestas (regla 11).

Cada entrada aceptada se traduce a una medida del perfil y la recarga ( `report --top-up` ) la mide sobre `build` contra su propio nulo y **vuelve a juzgar el mapa entero**. Últimas tres corridas ( `runs.jsonl` ):

| día | papers | rechazados | entradas nuevas | tokens |
|---|---|---|---|---|
| 06-10 | 21 | 14 | 7 | 204.392 |
| 07-10 | 31 | 24 | 12 | 467.303 |
| 08-10 | 20 | 17 | 3 | 192.573 |

Lo que el código no puede comprobar, y lo dice su README: que el valor de un campo no diga más que la cita que lo ancla.

### 3.5 El tablero ( `studies/research/board/` )

**3.5.1 La puerta** ( `many.py:31-39` ). Parte de `scores.csv` sin las familias de reloj. Una celda entra si:

- tu prior la da por **Alta** —Alta en la matriz de `prior.yaml` , o familia principal o secundaria de ese marco ( `prior.py:11-25` )—, **o**
- pasa los cuatro filtros desnuda, **o**
- una variante del barrido pasa en meseta.

El «pullback» de tu prior no es una de las siete familias: se archiva bajo `reversion` y la página lo marca. USDCHF, UKOIL y USOIL no tienen prior.

**3.5.2 La evidencia, graduada** ( `factors.py:51-66` ):

| nivel | valor | cuándo |
|---|---|---|
| desnuda | 1,00 | la familia pasa los cuatro filtros |
| meseta | 0,80 | una variante del barrido pasa en meseta |
| en contra | 0,00 | el líder es significativo y paga menos de 0,5×; o p cruda ≥ 0,95 con múltiplo ≤ −1; o el barrido no llega a 0,5× con ninguna variante |
| débil | 0,50 | el líder es significativo y estable (y no es ninguno de los anteriores) |
| ninguna | 0,25 | el resto |

**3.5.3 Los puntos** ( `factors.py:69-73` , pesos en `board/config.yaml:8-13` ):

```
puntos = 100 × freno × (×0,5 si «en contra») ×
         (0,30·prior + 0,30·evidencia + 0,10·señal + 0,15·hueco + 0,15·pasado)

prior   Alta 1 · Media 0,5 · Baja o ninguna 0
señal   múltiplo ÷ 10, con tope en 1; 0 si la celda entra sólo por la prior («sin medir»)
hueco   1 / (1 + corridas de esa familia en esa celda)
pasado  la tasa encogida del apartado 2.9
freno   1 / (1 + ideas ya gastadas en la celda ÷ 6)
```

**3.5.4 El ejemplo real, a mano.** DJ30 · H1 · largo · ruptura:

```
el 2-10, sin ideas gastadas:
  0,30×0,5 (Media) + 0,30×1 (desnuda) + 0,10×0,314 + 0,15×1 + 0,15×0,5 = 0,7064  →  70,6 puntos,
puesto 1
hoy, con las tres ideas de la propuesta ya escritas:
  freno = 1 / (1 + 3/6) = 0,667     70,6 × 0,667 = 47,1 puntos, puesto 313
```

**3.5.5 El tablero de hoy** (recalculado por mí en memoria; no escrito en disco):

| puntos | celdas | qué son |
|---|---|---|
| 67,5 | 20 | prior Alta + evidencia débil. 18 de reversión, 2 de momentum; 5 marcadas «pullback» |
| 60,0 | 276 | prior Alta, sin evidencia medida |
| 54,8 | 1 | **EURUSD M15 corto momentum**: pasa desnuda, prior Baja. Puesto 297 |
| 52,5 – 40,0 | 19 | celdas con ideas gastadas o corridas previas (14 del oro a 51,4), y **DJ30 H1 largo ruptura** a 47,1, puesto 313 |
| 26,2 | 43 | prior Alta, **medido en contra** |

Entran 359 de 1.064: 357 por la prior, 2 por desnudas, 0 por el barrido. 22 llevan la marca de costes provisionales.

**La página imprime 40 líneas** ( `config.yaml:29` , `page.py:48` ): las 20 de 67,5 y las 20 primeras de 60,0. Los empates se deshacen por múltiplo (todas 0) y luego por orden alfabético del activo y del marco ( `many.py:67` ). AUDJPY ocupa 20 de las 40 líneas.

**3.5.6 El autoexamen** ( `selfcheck.py` ). Cuando haya 20 corridas cerradas, parte por la mediana de la puntuación del perfil y compara la tasa de supervivencia de las dos mitades (Fisher, 10 %). Hoy: `{"enough": false, "closed": 0, "needed": 20}` .

### 3.6 El `ideaExpert` ( `.claude/agents/ideaExpert.md` )

Agente Opus con todas las herramientas. Lo que su ficha le exige, con la línea:

| exigencia | línea | ¿lo comprueba el código? |
|---|---|---|
| una condición de entrada, sencilla, con razón económica | 18-20 | no |
| un mecanismo: quién está forzado a operar y por qué | 21-24 | no (lo ataca el revisor) |
| expresable con bloques de SQX, nativos o autorables | 25-26 | no aquí; en el paso 2 |
| operable tras costes; rechazar lo que mueva «unos pocos spreads» | 27-29 | no |
| distinta de lo ya probado | 30-31 | no (lo ataca el revisor) |
| leer la memoria de la celda antes de mirar datos | 35-40 | no |
| postura escrita ante cada fracaso cerrado | 42-47 | **sí** ( `proposal.py:37-43` ) |
| hoja de hipótesis con sus cuatro líneas | 49-67 | **sí**, que existan y que los dos números sean > 0 ( `sheet.py:20-30` ) |
| medir sólo en `build` | 82-86 | no |
| estadísticas condicionales rápidas, nunca una optimización | 85-86 | no |
| contar cada hipótesis medida y escribir la cifra | 87-88 | no; la memoria la lee del texto |
| cada lectura ambigua, listada y fijada (regla 11) | 91-97 | sólo que una pregunta traiga al menos dos lecturas ( `proposal.py:87-88` ) |
| una dirección por idea (regla 13) | la ficha no la enuncia (sólo pregunta el lado como ejemplo de ambigüedad, 91-97) | **sí**, la de la celda ( `proposal.py:78-80` ) |

**Lo que hizo en el caso real** (fichero del DJ30): declara que todo se midió en `build` (30-09-2013 a 31-12-2019, 30.246 barras H1) y que `oos1` y `oos2` no se leyeron; mide con una posición a la vez, contra la media incondicional del mismo horizonte; cuenta **68 estadísticas condicionales agrupadas en 21 hipótesis**; y dice de la idea C que la evidencia es floja y que la entrega porque se le piden tres mecanismos. Sus mediciones no llevan nulo ni corrección: son «× coste», «t del exceso» y «n de 7 años».

### 3.7 El `ideaReviewer` ( `.claude/agents/ideaReviewer.md` )

Agente Opus con sólo `Read` , `Grep` y `Glob` (línea 5): no puede ejecutar nada, y por eso su revisión no añade miradas. Cuatro ataques obligatorios por idea (líneas 45-64): mecanismo, coste, parecido con lo ya probado, ambigüedad. Un veredicto de tres: `resiste` , `dudosa` , `tumbada` .

**El caso real.** A la idea A le ataca la simetría («compresión y luego expansión» predice igual la ruptura a la baja, y el lado corto no funciona: quien paga puede ser la deriva del índice), calcula con la ficha que el coste en `oos1` es un 48 % mayor que en `build` (7,7 puntos frente a 5,21), señala que A y B comparten el disparador exacto y que el 86 % de las rupturas del `build` ya estaban sobre la media de 200, y deja abiertas todas las lecturas. Veredicto: dudosa.

### 3.8 El `researchDirector` y la propuesta ( `board/proposal.py` )

**Lo que `check` rechaza** ( `proposal.py:46-90` ), una frase en castellano por cada fallo:

| negativa | línea |
|---|---|
| falta `cell`, `diagnosis`, `measures`, `ideas` o `prior_failures` | 61-65 |
| sin revisor, o el revisor es quien propone (`ideaExpert`, `researchDirector`) | 66-67 |
| un fracaso cerrado de la celda sin postura (`adopta`/`rechaza`) o sin razón | 37-43, 68 |
| no son tres ideas con tres nombres distintos | 70-71 |
| a una idea le falta un campo de los doce | 77 |
| la dirección de una idea no es la de la celda (regla 13) | 78-80 |
| la cantera no es `perfil`, `libros` ni `propia` | 81-82 |
| la paleta del hueco usa la familia de la condición fija o una «igual» (ruptura, tendencia y momentum entre sí) | 83-86 |
| una pregunta con menos de dos lecturas (regla 11) | 87-88 |
| hoja de hipótesis incompleta, o revisión sin veredicto o con un eje vacío | 89 |

**Lo que `check` no mira:** que la celda esté en el tablero (eso lo hace `stamp`, que aborta si no está, `proposal.py:114-116`); cuántas condiciones lleva la regla (regla 14); si usa reloj o calendario (regla 15); si las tres ideas son tres mecanismos o una con tres periodos; si las hipótesis medidas están contadas; si se leyó algo fuera de `build`.

**Lo que `stamp` añade** (`proposal.py:109-123`) para que el agente no lo invente: el identificador, la fila de la celda en el tablero, «ya van K ideas», la marca de costes provisionales, el sello de cada hoja, si el veredicto del revisor quita o sólo anota, y dos costes fijos:

> «Las supervivientes se parecerán: tres ideas de la misma familia en el mismo mercado ganan y pierden los mismos años; para la cartera cuentan como poco más de una.» · «El listón de la celda sube tres ensayos de golpe.»

**Cuándo una idea es lanzable** (`proposal.py:98-106`): no vetada, sin preguntas abiertas, y —sólo si `review.removes` estuviera en `true`— no tumbada.

**El caso real.** La propuesta guardada dice en su diagnóstico «la celda n.º 1 del tablero (70,6 puntos)» y, sellado por Python, «puesto 311, 47,1 puntos, ya van 3 ideas»: el sello contaba como gastadas las tres ideas de la propia propuesta. Se corrigió el 4-10 (`proposal.py:151-154,169`, OPEN #101): con el código de hoy esa misma propuesta se sellaría en el puesto 1 con 70,6 y 0 ideas (comprobado por mí).

### 3.9 Las reglas duras, y dónde actúa cada una en este paso

| regla | en el paso 1 | más adelante |
|---|---|---|
| **11** ambigüedad = pregunta | fichas de los tres agentes; `check` exige ≥ 2 lecturas por pregunta; `launchable` bloquea la idea con preguntas abiertas | — |
| **13** una dirección | `check` (la de la celda); el perfil mide largo y corto aparte; filtro `two_legs` del scout | `sqx.projects.builder.directions` (`builder.py:34-42`) se niega a una plantilla con las dos entradas |
| **14** ≤ 2 condiciones por lado | sólo en la cola de bibliografía (`conditions`); el `ideaExpert` la aplica de palabra (A y B: «dos fijas, sin hueco») | `buildrules.what_to_build` y `over_the_cap`, llamados por `builder.py:219`; tope en `assets/_build.yaml:24-25`. Cuenta condiciones por la estructura de la plantilla (un bloque compuesto cuenta 1), no por la prosa de la idea; no ejecutado contra SQX |
| **15** sin reloj, sin stop/limit, sin paleta entera | las medidas de reloj no lideran ni entran al tablero; filtros `clock` y `not_sqx_at_market` del scout; `check` prohíbe que la paleta repita la familia fija | `sqx/projects/buildcensus.py` clasifica los bloques de reloj del build |

---

## 4. El criterio cargado y por qué

### 4.1 Tal como está hoy en disco

No hay clave del paso 1 en `pipeline/autopilot/criteria.yaml`. Lo que funciona como criterio:

| qué | valor | dónde | de dónde sale |
|---|---|---|---|
| FDR del mapa | 5 %, BH sobre todas las pruebas | `marketProfile/config.yaml:26` | diseño del director, 1-10 |
| sorteos del nulo | 3.000 | `:13` | medido el 2-10: con 1.000, la p mínima era la única que BH nombraba |
| bloque del nulo | 1 barra | `:23` | medido el 1-10 ( `blocklen.py` ) |
| paga | ≥ 2× el coste de ida y vuelta, bruto | `:27` | diseño |
| estable | signo en más de 0,5 de los años | `:28` | diseño |
| frecuente | ≥ 40 al año; suelo 35 por activo | `:31` , `many.py:10` | tuyo, 2-10 |
| mínimo para contrastar | 30 operaciones | `:33` | diseño |
| puerta del tablero | prior Alta **o** desnuda **o** meseta | `board/many.py:36-39` | tuyo, 2-10: «usa las dos» |
| pesos del tablero | 0,30 · 0,30 · 0,10 · 0,15 · 0,15 | `board/config.yaml:8-13` | elegidos para que «prior Alta sola» y «desnuda sola» queden comparables |
| freno | a la mitad con 6 ideas | `:39` | diseño |
| tres ideas por propuesta | 3 | `proposal.py:70` | diseño del director |
| el revisor quita o anota | anota ( `false` ) | `memory/config.yaml:172` | tuyo |
| tolerancia de la hoja | factor 2 | `memory/config.yaml:166` | encargo 42 |

### 4.2 Por qué el resultado es «2 de 1.064»

Con estos datos no se puede decir si los mercados están vacíos o no en tendencia, ruptura y momentum: el número de celdas que pasan (2) es el que daría el azar (apartado 5.15). Lo que sí se ve es la combinación de dos exigencias que tiran en sentidos opuestos (apartados 3.1.8 y 3.2):

- para salir **significativa** entre 16.036 pruebas hace falta un z de 3, y eso lo consiguen sobre todo reglas con muchísimas operaciones (reversión a muy corto plazo), cuyo efecto por operación es una fracción del coste;
- para **pagar** 2× el coste hacen falta reglas más selectivas, con menos operaciones, que entonces no llegan a z = 3.

El hueco entre las dos lo mide la potencia: en H1, una ventaja real de 2× el coste se vería, como mucho, en el 14-36 % de las pruebas para las cinco familias de la tabla (sesión 8 %; patrón no estimada). Es un techo: viene de una aproximación que, donde se pudo simular (H4), promete más de lo que la simulación detecta, y es del corte del 2-10.

### 4.3 Cuánto cambia el resultado si mueves una pieza

Todo calculado por mí sobre `measures.csv` de hoy, sin volver a medir nada:

| cambio | celdas con medida que pasa | cuáles se añaden |
|---|---|---|
| lo cargado | 2 | — |
| sacar de la familia de BH las 2.178 pruebas que no se contrastaron (p = 1 puesta a mano por menos de 30 operaciones) | 2, las mismas | ninguna: N = 13.858, corte 5/3001, 528 nombradas (calculado por el revisor sobre `measures.csv`) |
| además, quitar las 454 que se contrastaron y dieron p = 1 (o sacar todas las 2.632 con p = 1) | 5 | EURUSD M30 corto `day_down_volhigh_1d` (8,1×), USDJPY M30 largo `bar2atr_trail` (3,3×), XAUUSD M30 largo `bar2atr_trail` (2,8×), las tres con p = 6/3001. Con las 2.632 fuera, N = 13.404 y 553 nombradas. Es descartar pruebas por su resultado (las que fueron al revés) y sesga a favor; no se propone |
| corrección por familia (ya calculada, `passes_family`) | 3, y son otras | las tres de reversión del apartado 3.1.8; las dos oficiales salen |
| bajar la frecuencia a 35 | 2 | ninguna: las tres que caen por frecuencia operan 15, 13 y 5 veces al año |
| quitar la significancia y dejar sólo paga + estable + frecuente | 267 medidas | de ellas, 67 con p cruda ≤ 0,05; mediana de p, 0,13 |

La última fila es la que dice cuánto filtra la estadística: sin ella, el mapa señalaría 267 medidas, casi todas indistinguibles del azar.

Ninguna medida pasa en todas las filas de esta tabla: entre todas suman 7 medidas distintas y no comparten ninguna (calculado por el revisor; apartado 5.18). El tamaño de la familia sí cambia el corte (4/3001 con 16.036, 5/3001 con 13.858).

### 4.4 Cómo sería una idea «buena» al salir del paso 1, con lo que hay

Con lo que el código y las fichas piden hoy: la celda está en el tablero; la regla es exacta y de una dirección; cada lectura dudosa está contestada por ti; trae mecanismo, hoja de hipótesis sellada y la postura ante los fracasos previos; el revisor la atacó por los cuatro frentes; y sus miradas están contadas. **Nada de eso es evidencia de que funcione.** Es evidencia de que, si falla, se sabrá por qué, y de que lo que venga después no se contaminó.

---

## 5. Lo que no cuadra o no está medido

### 5.1 Las hipótesis medidas de la propuesta del DJ30 cuentan cero

`ideaExpert.md:116` pide el apartado «**Hipótesis medidas en total** (el número)». `memory/ideas.py:11` busca `Hip[oó]tesis medidas:?\s*\**\s*(\d+)`: la palabra «medidas», dos puntos opcionales, y el número. El fichero del DJ30 escribe «## 3. Hipótesis medidas en total» y la cifra en la línea siguiente («**68 estadísticas condicionales**»): no casa. Es peor que «no casa con el título»: el patrón exige un dígito justo después de «medidas», así que ni «Hipótesis medidas en total: 68» casaría (le sigue «en total»). Un fichero que siga a la letra `ideaExpert.md:116` (`3. **Hipótesis medidas en total** (el número)`) nunca se cuenta; el oro sólo cuenta porque su autor escribió «Hipótesis medidas: **24**» (línea 223) y «**29**» (línea 340; el lector toma la mayor). Resultado: `hypotheses_measured` vacío en las tres ideas, `ideas_spent.csv` dice 0 hipótesis para DJ30 H1 largo, y `trials.spent("DJ30","H1",…)` devuelve 0.

Además, aunque casara: el fichero da dos cifras, 68 estadísticas y 21 hipótesis. La ficha no dice cuál es «la mirada». Es una ambigüedad sin resolver.

### 5.2 Contar las miradas no mueve el listón

La N de un estudio son miles de estrategias puntuadas; sumar 29 o 68 no la cambia cuando N son miles; con una N de decenas sí (el test del repo, `knowhow/market-profile/ideas-are-ledger-trials.md` y `tests/test_ledger_ideas.py`: DSR 0,4025 → 0,3764 con 29 sobre N = 50), y la N de una plantilla sin hueco aleatorio (las ideas A y B) no está medida. Medido el 2-10 (`knowhow/market-profile/ideas-are-ledger-trials.md`): N de 15.967 a 15.996, listón de 0,083981 a 0,083990. Mi cuenta coincide: √(2·ln 15.967) = 4,3996 y √(2·ln 15.996) = 4,4000.

La cuenta es contabilidad honesta, pero **la selección de la idea no se corrige en ningún sitio**: ni las 68 miradas del experto, ni las 211 pruebas del perfil en la celda (se corrigen entre ellas, pero la celda elegida es precisamente la que pasó), ni las variantes del autor de un paper. Lo que protege de esa selección no es un número de este paso, sino que los pasos 7 en adelante usan datos que la idea no vio.

### 5.3 La página del director hoy no tiene nada medido

Apartado 3.5.5. Con el orden actual, una celda que pasa los cuatro filtros con prior Baja o Media (54,8 – 70,6) queda por debajo de 276 celdas de prior Alta sin evidencia (60,0) en cuanto tiene ideas gastadas o su prior es Baja. El comentario de `board/config.yaml:4-7` dice que «prior Alta sola» y «desnuda sola» quedan «comparables»: lo son en puntos, pero al haber 276 empatadas a 60,0, «comparable» significa quedar fuera de la página. Y entre las 276 el orden es alfabético. Calculado por el revisor: una celda que pasa desnuda con prior Baja puntúa 52,5 más el múltiplo, así que necesita pagar 7,5× el coste para empatar con una de prior Alta que nadie ha medido («sin evidencia» suma 7,5 puntos). Y las 20 líneas de cabeza son, por definición (`factors.py:51-66`), medidas significativas y estables que no pasan los cuatro filtros: de las 276 nombradas, 222 pagan menos de 0,5× y 49 entre 0,5× y 2×. Son el grupo que el punto 4 del resumen describe como significativo y no rentable.

`researchDirector.md:20-32` dice que lee «una página, no el repositorio». Hoy esa página son 20 celdas de reversión/momentum «débiles» y 20 de AUDJPY.

### 5.4 El veredicto del mapa depende de cuántas reglas haya traído el scout

Apartado 2.4 y punto 3 del resumen. Cada medida nueva de bibliografía (no cada entrada: hay entradas en cola, con pregunta al dueño o que duplican una medida) añade 152 pruebas (2 lados × 76 celdas) a la familia de BH; entre el 2 y el 8-10 se añadieron 11 medidas (de 33 a 44), 1.672 pruebas (calculado por el revisor sobre `runs.jsonl` y `measures.csv`). Hechos: 14.364 → 16.036 pruebas en seis días; corte 5/3001 → 4/3001; q del DJ30 0,030 → 0,033. Calculado por mí: con su puesto fijo en 486, la celda deja de pasar cuando N > 24.308, es decir, tras unas 54 medidas más que no aporten ninguna p pequeña.

No es un error de BH: es que «la familia» no está definida. ¿Son todas las reglas que se hayan medido alguna vez, o las de una tanda? Nada lo fija, y `profilePower` y `pooledProfile` se calcularon con el corte del 2-10.

### 5.5 Las pruebas no contrastadas cuentan en el denominador

2.178 filas tienen p = 1 puesta a mano porque no llegaron a 30 operaciones (`one.py:151,159`). No son pruebas, pero BH las cuenta en N (`many.py:70` corrige todas las filas). Sin ellas (calculado por el revisor): N = 13.858, corte 5/3001, se nombrarían 528 en lugar de 504 y pasarían las mismas 2 celdas: el veredicto no cambia (2 → 2). Es conservador, no incorrecto. Las otras 454 filas con p = 1 sí se contrastaron (apartado 3.1.2); quitarlas además daría 5 celdas, pero es descartar pruebas por su resultado y no se propone.

### 5.6 La frecuencia cuenta operaciones solapadas en las medidas originales

`measure/trades.py:22-23` abre una operación en cada barra con señal. El fichero de idea del DJ30 lo midió: `channel55` da 145 operaciones al año en el perfil y **95** con una posición a la vez (cifra del `ideaExpert`, no mía). El filtro de 40 al año se aplica a la primera cifra; una estrategia de SQX hace la segunda, o menos tras el hueco aleatorio. Las medidas `rule` no tienen este problema.

### 5.7 «Estable» cuenta el signo, no el coste, y un trimestre vale un año

Apartado 3.1.6: 2014 y 2015 del DJ30 dan menos que el coste y votan a favor; y en las medidas vecinas el último trimestre de 2013 es un año entero de siete.

### 5.8 El efecto es bruto y sin swap; el coste es el de `build`

El «× coste» no descuenta el swap (`inputs.py:81-82`). Para el DJ30 el propio `ideaExpert` lo cifra en unos 3,5 puntos por noche, 0,67 costes. Y el coste es el del tramo `build`; el revisor calculó que el de `oos1` es un 48 % mayor. Una celda que paga 2,0× en `build` paga 1,35× con el coste de `oos1`.

### 5.9 La ficha del revisor pide 40-50 operaciones al año

`ideaReviewer.md:55-56`: «the owner's floor of 40-50 trades a year». El suelo de trabajo es 35 desde el 2-10 y el filtro del perfil 40 (`many.py:10`, `config.yaml:31`).

### 5.10 «Sólo `build`» y «nunca una optimización» no se pueden comprobar en el experto

El perfil lo fuerza en código y con un test. El `ideaExpert` escribe scripts sueltos en `scratch/`; nadie guarda qué leyeron. Y `ledger.gate` sólo se activa con `ALGO_AUTONOMOUS=1` (`WORKFLOW.md`, «Qué segmento toca cada paso»). En el caso real, el umbral «9,0 × ATR» de la idea A salió de probar terciles y umbrales de 7 a 10 sobre `build`: es una elección de parámetro dentro de muestra, pequeña pero real, y la ficha dice «nunca una optimización» sin decir dónde está la raya.

### 5.11 El hueco y el pasado no ven las corridas de Estrategia Simple

34 de las 44 corridas de la memoria no tienen dirección ni familia, así que no cuentan en el «hueco» de ninguna celda (`board/inputs.py:61-64` exige la celda exacta) ni en el «pasado». Las seis corridas del NIKKEI225 M30, que son las que más lejos han llegado en la cadena, no existen para el tablero. Una **idea** sin familia sí cuenta contra todas las familias de su celda (`inputs.py:49-58`); una **corrida** sin familia, contra ninguna.

### 5.12 `WORKFLOW.md` no describe este paso

Fila 1: «Idea en el chat | — | —» (línea 26; lo mismo en `ui/daemon/workflow/steps.py:23`). Y «`build` — sólo el paso 6» (línea 141), cuando el perfil, el barrido, la potencia, el agrupado y el `ideaExpert` leen `build` antes.

### 5.13 Textos con cifras de antes

- `marketProfile/README.md`: «189 tests per cell, 14,364 in the map», «99 measures». Hoy 211, 16.036 y 110.
- Manual, capítulo `82-perfil-de-mercado.md`, líneas 75 y 94: «14.364».
- `profilePower/config.yaml:31`: «0.00167 on 2026-10-02»; hoy el corte es 0,00133.
- `series.py:137`: «the same bars, in blocks, in another order». Es con reposición (`draws.py:85`).
- `proposal.py:145`: «the director's five steps»; son seis (`STEPS`, líneas 15-17).

- `AlgoData/research/board/board.json` y `memory/*.csv` son del 2-10; el tablero se recalcula cada vez, pero quien abra el fichero ve el orden viejo. El CSV de corridas guardado (`attempts.csv`, del 2-10 a las 20:27) tiene 62 filas y la memoria da hoy 44. Por nombre de proyecto, desaparecieron 34 (los `Test_Calib_USDJPY_*` y `TestUSDJPY_Workflow_v1`, borrados el 2026-10-04) y hay 16 nuevos (cinco `Test_NIKKEI225_M30_simpleL_*`, `Test_EURUSD_M30_filterMALong_10k`, `Test_USA500_H1_indicesIBS`, `Test_USDJPY_M30_doubleROCLong_*`, etc.): 62 − 34 + 16 = 44 (calculado por el revisor). Los 34 «sin dirección ni familia» de hoy son otro conjunto. El CSV se rehace con cada `memory.report`.

### 5.14 Lo que no está medido

1. **Si el perfil guía.** El autoexamen necesita 20 corridas cerradas; hay 0. No se sabe si una celda «desnuda» da más supervivientes que una de prior sola.
2. **Si tu prior guía.** Lo mismo. 357 de 359 celdas entran por ella, y 43 de esas están «medidas en contra».
3. **Cuántas falsas alarmas da el `ideaExpert`.** No hay control: nadie le ha dado una serie sin nada dentro para ver cuántas «ideas con evidencia» devuelve.
4. **Si el revisor acierta.** Ninguna idea revisada se ha construido.
5. **Si la hoja de hipótesis predice.** Ninguna hoja sellada tiene build.
6. **La potencia con el corte de hoy** y con el mapa de 16.036.
7. **El tamaño real de los nulos de la recarga.** Las medidas añadidas después se contrastan contra sorteos propios, con otra semilla, distintos de los del resto de su celda (`topup.py`); no he comprobado si eso altera algo.
8. **Las falsas alarmas del nulo al nivel en que decide (0,13 %), y en las 79 medidas nuevas.** La comprobación del apartado 2.2 es al 5 % y sólo cubre las 31 medidas originales.

### 5.15 Las dos celdas que pasan salen de un grupo sin exceso de p pequeñas

Todo en este apartado, salvo lo dicho del código, es calculado por el revisor sobre `measures.csv`. Entre las pruebas con operaciones, contrastadas y sin reloj de tendencia + ruptura + momentum (6.841; las dos oficiales son de ruptura y de momentum) hay **7 con p ≤ 4/3001** y el azar solo daría 9,1. En reversión + patrón hay 360 donde el azar daría 6,8. BH aplicado sólo a esas 6.841 (o a las 6.993 con las no contrastadas) no nombra ninguna. Con `q_family` (ya en el CSV) tampoco pasa ninguna de las dos, cosa que el apartado 3.1.8 dice sin sacar la consecuencia. El «puesto 486» que da q = 0,033 al DJ30 son casi todo pruebas de reversión, patrón, volatilidad y sesión: las 504 nombradas son 25 nombres de medida (`run3_fade` 112, `range_acf` 76, `best_hour` 48). Es decir, las dos celdas pasan porque la corrección toma prestado el rango de la reversión (mediana 0,28× el coste en las nombradas). Además, hay 2.864 pruebas frecuentes en las que un z de 3 ya vale 2× o más el coste; por puro azar cruzarían p ≤ 4/3001 unas 3,8, y han pasado 2: que pasen 2 no distingue el mapa de un mapa sin nada dentro.

### 5.16 La p de una celda en el corte es ruido de sorteo

Calculado por el revisor. El DJ30 tiene 2 nulos de 3.000 por encima del real; hoy se nombra hasta 3. Con 2 observados, el intervalo del 95 % de la p verdadera va de 0,00008 a 0,0024 (el corte, 0,00133, está dentro). Si la p verdadera fuera 0,001, volvería a pasar el 65 % de las veces; si fuera exactamente el corte, el 44 %; si fuera 0,002, el 15 %. Hay 41 pruebas en p = 3/3001 o 4/3001 en esa situación. La semilla es entropía del sistema en cada corrida (`config.yaml:16`, `seed: null`) y la recarga usa otra (`topup.py:20-21`). Volver a medir con otra semilla dejaría la celda fuera entre una vez de cada tres y una de cada dos: el veredicto de una celda en el corte no es reproducible.

### 5.17 El múltiplo que enseña lo que pasa viene inflado

Calculado por el revisor, con la aproximación normal sobre la fila de `measures.csv`: el error de la medida del DJ30 es 0,82 costes; el exceso observado sobre el nulo, 2,65× (el 16 % del t es deriva). Condicionado a pasar el corte (z ≥ 3,00): si el exceso verdadero fuera 2×, pasaría el 25 % de las veces y se vería 3,09×; si fuera 1×, pasaría el 3,3 % y se vería 2,92×; si fuera 0, se vería 2,82×. Lo observado (2,65×; z = 3,08, pegado al corte) es lo que se ve en los tres casos. El 3,14× de la tabla del apartado 3.1.6 no es una estimación de lo que pagará.

### 5.18 BH sin su supuesto, y las 16.036 pruebas no son 16.036

Calculado por el revisor sobre la columna `z`: la correlación del z entre la misma prueba del mismo activo en marcos contiguos tiene mediana 0,83 (máx. 0,94); largo frente a corto de la misma medida en la misma celda, 0,77; entre índices en H1, mediana 0,42 (DJ30-USA500 0,57); entre dos celdas cualesquiera, 0,38. Por autovalores, las 76 celdas equivalen a entre 4 y 30 independientes y las pruebas de una celda a entre 12 y 53: un orden de cientos a mil y pico de pruebas efectivas (estimación gruesa: la correlación mezcla dependencia y efectos reales compartidos). Las pruebas de una celda comparten además los mismos 3.000 sorteos (`one.py:82-86`). Con dependencia positiva BH sigue acotando la media, pero la proporción real de falsas en una tanda concreta puede ser 0 o casi todas: caen o aciertan en bloque (M15 y M30 del mismo activo). Que el recuento vaya de 2 a 5, a 3 distintas o a 0 según la familia es eso mismo visto desde fuera: sumando las variantes de la tabla 4.3, 7 medidas; en su intersección, ninguna. El dossier no decía el supuesto de BH (independencia o dependencia positiva y lista fijada de antemano) ni que ninguna medida sobrevive a todas las formas de contar.

### 5.19 «Sólo `build`» no corta la fuga por lo que ya se sabe de 2020-2026

(a) La prior que mete 357 de 359 celdas en el tablero se transcribe de un documento fechado el 2026-10-02 (`board/prior.yaml:1-3`), dos días después del fin de `oos2` (2026-09-30, `assets/_policy.yaml:81`). (b) La cola de bibliografía sólo compara la muestra del paper con `build` (`literature/selection.py:16-37`, campo `overlaps_is`); nada la compara con `oos1` ni `oos2`. De las 53 entradas con año final declarado, 13 llegan a 2020 o más (calculado por el revisor sobre los YAML de `AlgoData/research/literature/`); la idea C viene de un paper de febrero de 2025 y su entrada dice `sample: n/a`, `overlaps_is: n/a`. (c) El agente que propone conoce la literatura y los mercados posteriores a 2019. Una regla publicada en 2025 se publicó porque funcionó en una muestra que incluye los años de `oos1` y `oos2`: para esas ideas el retest en `oos1` no es del todo fuera de muestra, aunque ningún fichero de `oos1` se haya abierto. El apartado 2.7 sólo cuenta «las variantes que probaron los autores», no las fechas.

### 5.20 `proposal.check` no cuenta condiciones, el builder sí

Regla 14: `proposal.check` no cuenta las condiciones de una idea, pero el builder las cuenta y rechaza después (`buildrules.over_the_cap`, llamado desde `builder.py:219`; apartado 3.9). El coste de un fallo ahí es una propuesta construida a medias, no CPU.

---

## 6. Cierre

### 6.1 Lo que el paso exige hoy, en una tabla

| | forzado por código | pedido por la ficha de un agente | no pedido |
|---|---|---|---|
| la celda está en el tablero | sí | | |
| tres ideas, una dirección, la de la celda | sí | | |
| hoja de hipótesis, sellada | sí | | |
| revisión por otro agente, cuatro ejes | sí | | |
| postura ante cada fracaso previo | sí | | |
| preguntas con lecturas; sin contestar no se lanza | sí | | |
| mecanismo económico | | sí | |
| medido sólo en `build` | sólo el perfil | sí, el experto | |
| miradas contadas | | sí, y hoy mal leídas | |
| ≤ 2 condiciones, sin reloj | sólo en la bibliografía | sí | no en `check` |
| que la evidencia propia del experto supere un nulo | | | no |
| que «tumbada» impida lanzar | | | no (interruptor apagado) |

### 6.2 El resultado sobre lo que hay

Una propuesta, tres ideas, ninguna lanzable hoy: 40 preguntas abiertas. Si las contestaras, las tres serían lanzables, incluida la tumbada. Coste declarado: 0,8 horas de custodio cada una, dicho como estimación.

### 6.3 Las decisiones que te quedan

| # | decisión | qué ganas | qué cuesta |
|---|---|---|---|
| 1 | **Qué es una «mirada» del experto**: cada estadística (68) o cada hipótesis (21), y que el título del fichero y el lector de la memoria digan lo mismo (el lector exige un dígito justo tras «medidas»: o se arregla el lector, por ejemplo `Hip[oó]tesis medidas(?: en total)?\W*\**\s*(\d+)`, o la ficha pide literalmente la línea «Hipótesis medidas: N»; con la cifra en la línea siguiente, como el DJ30, ninguno de los dos basta: hay que fijar el formato) | la cuenta deja de ser 0 | nada en el listón (5.2); es contabilidad |
| 2 | **Si «tumbada» quita la idea** ( `review.removes` ) | no gastas 0,8 h de custodio y un ensayo de la celda en lo que el revisor ya tumbó | el revisor no está validado (5.14.4): puede tumbar una buena. En el caso real quitaría 1 de 3 |
| 3 | **Qué familia de pruebas corrige BH**: todo lo medido alguna vez (hoy), las pruebas de verdad contrastadas, o por tanda | un veredicto que no cambie cada mañana (sacar las 2.178 no contrastadas deja las mismas 2 celdas, pero el corte pasa de 4/3001 a 5/3001) | con una familia más pequeña, más falsos positivos; y hay que rehacer potencia y agrupado con el corte nuevo |
| 4 | **El orden del tablero cuando la prior y lo medido no coinciden**: hoy 276 celdas de prior sola empatan por encima de la única celda medida libre | que el director vea lo medido | bajar la prior va contra tu «la estadística es un extra, no un requisito»; subirla deja la página como está |
| 5 | **Cómo se desempata entre celdas iguales** (hoy, alfabético) | que AUDJPY no ocupe media página | hay que elegir un criterio: coste sobre ATR, cobertura, rotación de activos |
| 6 | **Si las corridas sin plantilla cuentan en el hueco y en el pasado** | el tablero sabría de 34 corridas que hoy no ve | hay que decidir a qué familia se apunta una Estrategia Simple |
| 7 | **El suelo de operaciones en la ficha del revisor** (40-50, 40 o 35) | una sola cifra en los tres sitios | ninguno |
| 8 | **Si «estable» debe pedir pagar el coste y no sólo el signo**, y si «frecuente» debe contar una posición a la vez | filtros que digan lo que parece que dicen | con seguridad menos celdas, y hoy son 2. No sé cuántas: habría que volver a medir |
| 9 | **Si quieres un control del experto**: una serie sin dirección, y ver cuántas ideas «con evidencia» devuelve | sabrías cuánto vale su «paga, 7/7 años» | una sesión Opus por control |
| 10 | **Contestar o vetar las 40 preguntas del DJ30** | la primera propuesta se construye y empiezan a existir corridas cerradas con familia | 2,4 h de custodio si lanzas las tres; el listón de la celda sube tres ensayos |
| 11 | **Poner el paso 1 en `WORKFLOW.md`** con lo que hay | la espina dorsal deja de decir «—» | ninguno |
| 12 | **Si `prev_day_break` (rotura del máximo o mínimo del día anterior) cuenta como bloque de calendario de la regla 15.** Hoy | saber si el filtro de reloj de `many.py:31` debería haberla quitado | si cuenta, hay que marcarla y rehacer el tablero; si no, nada |

| # | decisión | qué ganas | qué cuesta |
|---|---|---|---|
| | lidera la familia de sesión en 152 celdas y no está marcada de reloj ( `needs_clock` falso en las 1.064) | | |

### 6.4 El límite que no cambia

Todo lo que este paso mide está dentro de muestra. El perfil, el barrido, las miradas del experto: todos ven `build`, y el build de SQX verá lo mismo. Una idea que sale de aquí con buena cara no ha demostrado nada todavía; lo único que este paso puede hacer bien es **no estropear lo que viene después**: no tocar `oos1` ni `oos2` (sabiendo que la prior, la bibliografía y el propio agente ya traen información de esos años, apartado 5.19), dejar escrito qué se miró, y fijar por adelantado qué resultado mataría la idea.